

Literature Review

ZeroTrustCloud: Implementing and Automating Identity-Centric Zero Trust Architecture for Cloud-Native Environments

Submitted as part of Week 1 Milestone

Name: BAIRE GOWDA

Reg No: 25PG00157

1. Introduction

Cloud-native systems today are built very differently from the applications security models were originally designed around. Workloads move between clusters, scale up and down constantly, and are often spread across more than one cloud provider at once, which means the traditional idea of a trusted internal network with a firewall around it doesn't really apply anymore. Because of this, the idea of zero trust, where every request has to be verified based on identity rather than assumed to be safe because of where it came from, has become the direction most modern cloud security research is heading toward.

For this Week 1 submission, I reviewed ten academic papers published between 2023 and 2026, chosen because each one connects to a specific problem or component described in the ZeroTrustCloud project statement, identity and access management, policy enforcement automation, service mesh security, continuous verification, CI/CD pipeline security, and observability/compliance. Rather than just summarizing what each paper says, I've tried to note what problem the researchers were actually dealing with, what they did about it, what they found, and how it connects back to what our group is building.

2. Reviewed Literature

1. A Zero Trust Architecture Model for Access Control in Cloud-Native Applications in Multi-Cloud Environments (NIST SP 800-207A)

National Institute of Standards and Technology (2023) | NIST Special Publication

Link: <https://csrc.nist.gov/pubs/sp/800/207/a/final>

What They Did: The original NIST SP 800-207 document laid out the general idea of zero trust, but it did not go into detail about how this should work once an organization is running workloads across more than one cloud provider at the same time. This follow-up document (800-207A) tries to close that gap. It builds a reference model where the deciding factor for access is not where a request is coming from on the network, but who or what is making the request, verified through a cryptographic identity. The model relies on application identity infrastructure, with SPIFFE named specifically as the mechanism for issuing that identity, and it places the actual enforcement at API gateways and sidecar proxies rather than expecting every individual application to implement this logic on its own.

What They Found: There isn't a traditional "experiment" in this document since it's a standard, not a study, but its main conclusion is still important: once workloads are spread across AWS, Azure, GCP, or a mix of on-prem and cloud, network position (subnet, VPC, IP range) stops being a trustworthy signal. Two workloads can sit in what looks like the same trusted network segment and still have no real reason to trust each other unless their identity is independently verified. The document also points out that this identity check has to happen on every single

request, not just once at login, because workloads in cloud-native systems are short-lived and constantly changing.

Why It Matters to This Project: For our project this is basically the rulebook we're trying to build a working version of. It's the reason our IAM Engine in ZeroTrustCloud is built around identity verification instead of firewall rules or IP allowlists. One thing I noticed while reading it is that this document only describes the model at a fairly abstract level and doesn't say much about how to handle performance overhead when every request needs identity verification, which is something we might have to figure out ourselves when we build our prototype.

2. A Systematic Literature Review on the Implementation and Challenges of Zero Trust Architecture Across Domains

Mushtaq, M. M. et al. (2025) | Sensors (MDPI), Vol. 25

Link: <https://www.mdpi.com/1424-8220/25/19/6118>

What They Did: Instead of building anything new, the authors of this paper went through 74 existing studies on zero trust from very different fields, healthcare systems, IoT networks, enterprise IT, and cloud platforms, and tried to see how complete each implementation actually was. They scored each study against what they consider the core building blocks of zero trust: identity verification, continuous monitoring, micro-segmentation, and automated policy enforcement, and put together something like a heatmap showing which pillars were being handled well and which ones kept getting skipped or half-done.

What They Found: The pattern that came out of this review was that almost nobody does all four pillars properly at the same time. A lot of systems are strong on identity checks at login but weak on continuous monitoring afterward, or they do network segmentation well but still enforce policy manually. The other big finding was around AI-based monitoring specifically: even when these systems correctly flagged suspicious behavior, administrators in the reviewed studies often couldn't tell why the system made that decision, and this lack of explanation made people less willing to trust the system's output even when it was technically right.

Why It Matters to This Project: Reading this actually shifted how I explain our own project now. ZeroTrustCloud isn't proposing something that's never been tried, it's trying to be one of the more complete implementations by covering identity, policy, service mesh, continuous verification, pipeline security, and observability together, instead of picking one and calling it done, which seems to be what most of the 74 reviewed papers did. It also made me think that whatever anomaly detection logic goes into our Continuous Verification Engine should explain its reasoning somehow, not just output a flag, otherwise we'd be repeating the exact weakness this review pointed out.

3. Implementing Zero Trust Architecture in Cloud-Native Environments: Challenges and Best Practices

Dommari, S., & Khan, S. (2025) | SSRN Electronic Journal

Link: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5259339

What They Did: This paper is less about theory and more about what actually goes wrong when a company tries to switch from old perimeter-style security to zero trust inside a cloud-native, microservices-heavy environment. The authors walk through common problem areas: identity sprawl (too many service accounts and roles that nobody is really tracking anymore), RBAC settings that don't match up between different environments like staging and production, and

service-to-service communication that often isn't authenticated at all because everything is assumed to be "internal" and therefore safe.

What They Found: What stood out to me in their findings is that almost none of the failures they describe are really about missing technology. Teams generally know they should apply least privilege, use continuous authentication, and segment their network properly, but they don't, because doing it properly takes more time and effort than doing it the easy way. So the paper's real conclusion is that a lot of zero trust failures come down to operational shortcuts rather than a lack of tools.

Why It Matters to This Project: This connects to our project in a pretty direct way. If our IAM Engine automates just-in-time privilege management and RBAC/ABAC enforcement instead of leaving it up to someone remembering to configure it correctly, we're removing exactly the kind of shortcut this paper blames for most real-world zero trust failures. It made me feel more confident that automation isn't just a bonus feature for ZeroTrustCloud, it's actually the whole point of building it this way instead of relying on manual policy management like older systems do.

4. Analysis of Role-Based Access Control and Service Account Misconfigurations to Identify Vulnerabilities in Kubernetes

IJERT Authors (2026) | International Journal of Engineering Research & Technology

Link: <https://www.ijert.org/analysis-of-role-based-access-control-and-service-account-misconfigurations-to-identify-vulnerabilities-in-kubernetes-ijertv15is060910>

What They Did: This one is a lot narrower in scope compared to the others, it's specifically about Kubernetes RBAC and service accounts, and how badly configured these usually end up being in practice. The authors went through role bindings and service account setups across clusters and checked them against basic least-privilege principles, essentially doing an audit-style analysis rather than building a new tool or system.

What They Found: What they found is something that honestly sounds small but has a big impact: most service accounts end up with way more permission than they need. A pod that only needs to read a single config map often ends up with a service account that could list secrets across an entire namespace, usually because someone copied an existing role definition instead of writing a properly scoped one. And because nobody goes back to clean these up later, permissions just keep accumulating as a cluster grows and more services get added over time.

Why It Matters to This Project: This was one of the more relatable papers for me because it describes something that could easily happen in our own project if we're not careful. It made it pretty clear that our IAM Engine can't just set up RBAC once during deployment and call it finished, there needs to be some kind of ongoing check built in, which actually ties this component to our Continuous Verification Engine. If we want to actually demonstrate zero trust properly during evaluation, I think we should show some kind of periodic audit of our RBAC setup, not just the initial configuration screen.

5. Reusing Policy-as-Code Across CI/CD and Kubernetes Admission Control: An Empirical Assessment of Governance Consistency

MDPI Authors (2026) | Computers (MDPI), Vol. 15, Issue 7

Link: <https://doi.org/10.3390/computers15070453>

What They Did: This paper looks at a problem that I honestly hadn't thought about before reading it, most teams write two separate sets of policy rules, one for their CI/CD pipeline and a different one for their Kubernetes cluster's admission control, even though both sets are often checking for the same kinds of things, like not allowing containers to run as root or blocking images from untrusted registries. Because these two rule sets live in different places and get updated by different people at different times, they slowly drift apart from each other. The authors built an actual working setup where a single OPA/Rego policy definition is used at both enforcement points, as a pipeline gate before the image is even built, and again as an admission control check before the workload is allowed to run in the cluster.

What They Found: Because they tested this empirically instead of just describing it in theory, they were able to show that reusing the same policy definition at both stages noticeably reduced the number of inconsistencies compared to keeping two separate rule sets. They did mention some friction though, mainly because the pipeline stage and the cluster admission stage don't always have access to exactly the same context or data, so the policy sometimes needs small adjustments to work properly in both places.

Why It Matters to This Project: Out of all ten papers, this is probably the one that maps most directly onto what we're building. Our Policy-as-Code Engine is meant to use OPA with GitOps-based workflows, which is basically what this paper implements and tests. Reading this made me think we should try to design our own OPA policies so they can be reused at more than one enforcement point, like both the pipeline and the cluster, instead of writing them separately for each, since that reuse is where this paper says the real governance benefit actually comes from.

6. Zero Trust Security Model Implementation in Microservices Architectures Using Identity Federation

arXiv Authors (2025) | arXiv preprint 2511.04925

Link: <https://arxiv.org/pdf/2511.04925>

What They Did: This paper is about a problem that's pretty specific to service mesh setups, how do microservices talk to each other securely when they might be spread across different clusters or even different cloud providers, without just assuming the internal network is automatically safe. The authors built a system where every microservice gets issued its own cryptographic identity through SPIFFE/SPIRE, a SPIRE agent verifies each workload as it starts up and hands it a short-lived identity document called an SVID, and then every connection between services goes through mutual TLS, meaning both sides have to prove who they are rather than just the client proving it to the server the way a normal website login works.

What They Found: They also worked out how to federate this trust across multiple clusters, so an identity that was issued in one cluster could still be trusted by services running in a completely different cluster, without needing to build a brand new trust relationship from scratch each time. In their setup, this meant a service in one region could securely talk to a service in another region while still following the same zero trust rules, rather than falling back to a less secure network-level trust just because the two clusters were in different places.

Why It Matters to This Project: This maps almost exactly onto our Service Mesh Security Engine component. What I took away from it is that just deploying something like Istio isn't enough by itself to be considered zero trust, the actual zero trust part comes from combining the mesh with a proper workload identity system like SPIFFE. If our own Service Mesh Security

Engine is going to be genuinely zero trust and not just "encrypted traffic between services," we probably need to show SPIFFE-based identity issuance as part of the design, not only mTLS on its own.

7. Continuous Compliance and Security Drift Prevention on Kubernetes Worker Nodes Aligned with CIS Standards

Vance, E., & Sharma, A. (2026) | ResearchGate preprint

Link: <https://www.researchgate.net/publication/399614451>

What They Did: This paper is about something called configuration drift, and I didn't fully understand how big a problem this actually is until reading it properly. Basically, when you first set up a Kubernetes worker node, you can make it compliant with the CIS Kubernetes Benchmark, which is a standard security checklist. But as time passes and patches get applied, or someone manually tweaks a setting to fix some unrelated issue, the node slowly drifts away from that original compliant state, usually without anyone noticing until the next scheduled audit, or worse, until something actually goes wrong. The authors built a system they call CCDP (Continuous Compliance and Drift Prevention) that pairs a declarative OPA-based policy baseline with a real-time, agent-based monitoring and enforcement engine using Falco, which watches worker nodes continuously instead of only checking at fixed audit intervals.

What They Found: When Falco detects that a node has drifted away from the OPA-defined baseline, the system automatically corrects it back to the compliant state without needing a human to step in. In their controlled test environment, this cut the mean time to remediation (MTTR) for drift incidents by 98.7 percent, and the cluster sustained compliance with 95 percent of the assessed CIS controls, compared to only 72 percent in a baseline cluster that was managed manually.

Why It Matters to This Project: This lines up almost word for word with the "drift detection and remediation" feature that's literally named in our own project's Continuous Verification Engine description. What I understood from this is that pairing a policy engine with a runtime monitoring tool isn't something we'd be making up on our own, it's already an established pattern, and since we're already planning to use OPA for our Policy-as-Code Engine, it probably makes sense to reuse the same OPA policies here instead of building a completely separate rule set just for drift checking.

8. ARGO-SLSA: Software Supply Chain Security in Argo Workflows

arXiv Authors (2025) | arXiv preprint 2503.20079

Link: <https://arxiv.org/pdf/2503.20079>

What They Did: This paper deals with software supply chain security inside CI/CD pipelines, specifically the question of how you actually prove a build artifact like a container image wasn't tampered with somewhere between being built and being deployed. A lot of pipelines just trust that if an image shows up in the registry with the right tag it's fine, which is exactly the kind of blind trust that's led to real supply chain attacks before. The authors combined two things to fix this, Sigstore tools (Cosign and Fulcio) to cryptographically sign every artifact as it's built, and SLSA, which is a framework that grades how trustworthy a build process is based on things like whether the build is reproducible and whether the source was properly verified. They built all of this directly into Argo Workflows, which is a Kubernetes-native CI/CD tool, so it runs natively inside the cluster instead of being added on afterward as a separate step.

What They Found: The setup they built only allows artifacts that meet a minimum SLSA maturity level to actually get promoted to deployment, so instead of a simple "is it signed or not" check, there's an actual graded bar that a build has to clear. Combined with an immutable log of build provenance (using a tool called Rekor), this let them trace exactly how any deployed artifact was built and verify it wasn't altered along the way.

Why It Matters to This Project: This one is useful for two of our components at once, our Pipeline Security Engine's artifact signing requirement, and our Continuous Verification Engine's workload attestation piece, since cosign and Sigstore are literally named in our own problem statement. I think if we implement something similar, using a minimum SLSA-level gate instead of a plain signed/unsigned check would make our project look a lot more thought-through when it's evaluated, since it shows we understand there are different levels of build trustworthiness, not just a yes or no answer.

9. Ambush from All Sides: Understanding Security Threats in Open-Source Software CI/CD Pipelines

Z. Pan, W. Shen, X. Wang, Y. Yang, R. Chang, Y. Liu, C. Liu, Y. Liu, & K. Ren (2024) | arXiv preprint 2401.17606

Link: <https://arxiv.org/pdf/2401.17606>

What They Did: This paper is different from most of the others on this list because it isn't proposing any new system at all, it's a large-scale measurement study. The authors wanted a real answer to a simple question, how bad is CI/CD pipeline security actually, in practice, not just in theory. To find out, they built a custom analysis tool and scanned over 320,000 GitHub repositories that had CI/CD pipelines set up, looking specifically for things like secrets being exposed to untrusted code, workflows granting far more permission than they actually needed, and the use of third-party actions that weren't pinned to a specific, fixed version, meaning the action's underlying code could silently change later without anyone reviewing it again.

What They Found: There wasn't a solution being tested here, the value of the paper is purely in how much it quantifies. They found these kinds of misconfigurations were extremely common across real repositories, not rare edge cases, and they also tried to estimate the potential blast radius if these weaknesses were actually exploited by an attacker. Their overall conclusion was that most of this isn't intentional negligence, it's that teams genuinely don't realize how exposed their pipelines are until someone actually goes looking, which is exactly what this study did at scale.

Why It Matters to This Project: This paper is basically the evidence behind why pipeline security is treated as one of the five core problems in our own project's problem statement, it's not a made-up concern, it's something measured across hundreds of thousands of real repositories. It also made me think our Pipeline Security Engine shouldn't just focus on one single fix like secret rotation, since this paper shows the actual attack surface is spread across permissions, secrets, and unpinned dependencies as three separate risks, not one combined problem.

10. A Reference Architecture for Governance of Cloud Native Applications

arXiv Authors (2023) | arXiv preprint 2302.11617

Link: <https://arxiv.org/pdf/2302.11617>

What They Did: This paper focuses on a problem that's more about process than technology, compliance frameworks like SOC 2, HIPAA, and FedRAMP have hundreds of individual controls that an organization has to prove it's meeting, and doing that through manual, periodic audits doesn't really work anymore once a cloud-native system is changing constantly. By the time a manual audit finishes, the system being audited has usually already changed again, so the results are basically outdated the moment they're produced. The authors proposed a reference architecture that connects observability data, meaning the logs, metrics, and traces that are already being collected for normal monitoring, directly to specific governance controls, so evidence for compliance can be pulled automatically instead of gathered by hand every few months.

What They Found: Their main conclusion was that continuous, automated evidence-gathering from telemetry that already exists can replace a lot of the manual audit work that companies currently do, and that this keeps compliance verification current in a way that periodic audits simply can't match once systems are changing on a daily or even hourly basis.

Why It Matters to This Project: Before reading this I honestly thought of observability and compliance reporting as two separate features we'd need to build for our Observability & Compliance Engine, but this paper made me realize they're actually meant to be the same system, the observability data itself is the compliance evidence, it's not a separate thing sitting next to it. I think this is a useful way to frame our final report too, describing our Observability Engine as continuous, automated evidence generation for whatever compliance framework we choose to reference, rather than just calling it a logging dashboard with a compliance tab bolted on.

3. Overall Takeaway

Going through all ten papers together, a pattern became pretty clear to me: none of them individually cover the full scope of what ZeroTrustCloud is trying to do. The NIST document and the systematic review give the overall justification for why identity-based zero trust matters, the SSRN and IJERT papers focus specifically on where IAM tends to break down in practice, the MDPI and ResearchGate papers back up the policy-as-code and drift-detection side of things using OPA, the SPIFFE paper backs up the service mesh identity piece, and the last three papers (ARGO-SLSA, the CI/CD measurement study, and the governance architecture paper) cover pipeline security, artifact integrity, and compliance reporting respectively.

What this tells me is that ZeroTrustCloud isn't trying to invent something completely new from scratch, it's trying to combine six areas that, based on this literature, are usually researched and implemented separately, into a single working system. That's actually consistent with what the systematic literature review (paper 2) pointed out, that most real-world zero trust implementations only cover parts of the full picture. If we can get all six components working together even at a basic level for our prototype, that alone would already put our project ahead of a lot of what's described as "typical" in these papers.